DEPI INDUSTRY DELIVERABLE — MODULE 2

Literature Review & Evaluation

Project: Automated UI Testing for an E-Commerce Website

1. Background & Tool Selection

In the rapidly evolving landscape of e-commerce, ensuring a seamless user experience is critical for customer retention and maximizing revenue. Automated UI testing plays a vital role in verifying that the visual and interactive elements of a web application function correctly across different browsers and platforms.

Several automated testing frameworks exist in the industry, including Cypress, Playwright, and Selenium. For this project, Selenium WebDriver was selected as the primary testing tool. Selenium remains an industry standard due to its robust cross-browser compatibility, extensive community support, and flexibility in integrating with multiple programming languages such as Python and Java. Unlike some modern tools that are limited to specific environments, Selenium's architectural design allows for comprehensive end-to-end testing of complex e-commerce workflows.

2. Architectural Pattern: Page Object Model (POM)

Maintaining large test suites can become challenging as the application scales. To address this, the project adopts the Page Object Model (POM) design pattern. POM is widely recognized in software testing literature as a best practice. It separates the UI element locators (the web page representation) from the actual test scripts. This separation of concerns significantly enhances code reusability, reduces duplication, and makes the automation framework highly maintainable when UI changes occur.

3. Feedback & Evaluation Summary

> **Lecturer & Evaluator Feedback:** The framework architecture effectively implements separation of concerns through POM. Initial feedback highlighted the need to strictly enforce dynamic explicit waits over static delays to eliminate test flakiness, and to strengthen verification state assertions across all critical navigation steps.

4. Suggested Enhancements

- **CI/CD Integration:** Incorporate GitHub Actions workflows to trigger automated test suite execution automatically on every code push or pull request.
- **Grid & Cross-Browser Execution:** Expand execution across Chrome, Firefox, and Edge in parallel using Selenium Grid or cloud testing tools.
- **Advanced Visual Reporting:** Integrate Allure or ExtentReports to generate graphical test reports with automatic screenshot capture upon failure.

5. Final Grading Criteria

ASSESSMENT DOMAIN	WEIGHT	EVALUATION FOCUS & DELIVERABLE EXPECTATION
Documentation	20%	Clarity and completeness of planning, requirements gathering, task distribution, and risk management.
Implementation & Architecture	40%	Clean code quality, correct implementation of Page Object Model (POM), reusable locators, and dynamic waits.
Testing & Assertion Quality	20%	Robustness of test scripts, dynamic assertions, edge case coverage, and stability without timing failures.
Final Presentation & Reports	20%	Quality of project demonstration, test outcome summary report, and professional team presentation.